SPEC TÉCNICA — FitCoach Pro

Versão: 1.0
Data: 17/06/2026
Metodologia: Schema-Driven Development (SDD)
Referência: PRD v1.0

SOBRE SDD — Schema-Driven Development

Neste projeto adotamos **SDD** como metodologia central. Isso significa:

1. **Schema primeiro** — O banco de dados é a única fonte de verdade. Nenhuma feature é codificada antes do schema estar definido, migrado e testado.
2. **Tipos gerados** — Os tipos TypeScript são gerados automaticamente a partir do

schema Supabase (`supabase gen types typescript`). Nunca criados à mão.

3. **Contrato de API pelo schema** — As Edge Functions e Server Actions do Next.js apenas orquestram operações definidas pelo schema. Sem lógica de negócio fora do banco.
4. **RLS é a segurança** — Row Level Security do PostgreSQL é a única camada de autorização. Nenhuma verificação de `user.role` no frontend.
5. **Ordem de trabalho:** `Schema → Migration → Types → Service Layer → UI`

ÍNDICE

---

1. ESTRUTURA DE REPOSITÓRIOS

```
github.com/seu-usuario/
├── fitcoach-pro-web/          #
```

```
Next.js 14 — Dashboard do Personal
├── fitcoach-pro-mobile/       # Expo
React Native — App do Aluno
└── fitcoach-pro-supabase/     #
Migrations, Seeds, Edge Functions
```

1.1 Estrutura — fitcoach-pro-supabase

```
fitcoach-pro-supabase/
├── supabase/
│   ├── migrations/
│   │   ├──
20260617000001_create_personals.sql
│   │   ├──
20260617000002_create_students.sql
│   │   ├──
20260617000003_create_exercises.sql
│   │   ├──
20260617000004_create_workout_plans.s
ql
│   │   ├──
20260617000005_create_workout_session
s.sql
│   │   ├──
20260617000006_create_assessments.sql
│   │   ├──
20260617000007_create_invoices.sql
│   │   ├──
```

```
20260617000008_create_files.sql
│   │   ├──
20260617000009_create_notifications.s
ql
│   │   ├──
20260617000010_rls_policies.sql
│   │   └──
20260617000011_functions_triggers.sql
│   ├── functions/
│   │   ├── invite-student/index.ts
│   │   ├── overdue-
invoices/index.ts
│   │   ├── workout-
reminder/index.ts
│   │   └── generate-
invoice/index.ts
│   ├── seed.sql
│   └── config.toml
└── README.md
```

1.2 Estrutura — fitcoach-pro-web

```
fitcoach-pro-web/
├── src/
│   ├── app/                        #
Next.js App Router
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
```

```
│   │   │   └── register/page.tsx
│   │   ├── (dashboard)/
│   │   │   ├── layout.tsx
# Shell com sidebar
│   │   │   ├── page.tsx
# Home / Overview
│   │   │   ├── students/
│   │   │   │   ├── page.tsx
# Lista de alunos
│   │   │   │   ├── new/page.tsx
# Novo aluno
│   │   │   │   └── [id]/
│   │   │   │       ├── page.tsx
# Perfil do aluno
│   │   │   │       ├──
workouts/page.tsx
│   │   │   │       ├──
assessments/page.tsx
│   │   │   │       └──
invoices/page.tsx
│   │   │   ├── exercises/
│   │   │   │   ├── page.tsx
# Banco de exercícios
│   │   │   │   └── [id]/page.tsx
│   │   │   ├── workouts/
│   │   │   │   ├── page.tsx
# Todos os treinos
│   │   │   │   └── [id]/page.tsx
# Editor de treino
```

```
│   │   │   ├── invoices/page.tsx
│   │   │   ├── files/page.tsx
│   │   │   └── settings/page.tsx
│   │   └── api/
│   │       └── webhooks/
│   │           └── stripe/route.ts
│   ├── components/
│   │   ├── ui/
# shadcn/ui components
│   │   ├── dashboard/
# Componentes específicos do
dashboard
│   │   ├── students/
│   │   ├── workouts/
│   │   └── exercises/
│   ├── lib/
│   │   ├── supabase/
│   │   │   ├── client.ts
# Browser client
│   │   │   ├── server.ts
# Server client (SSR)
│   │   │   └── middleware.ts
│   │   ├── services/
# Service layer (SDD)
│   │   │   ├── personal.service.ts
│   │   │   ├── student.service.ts
│   │   │   ├── workout.service.ts
│   │   │   ├── exercise.service.ts
│   │   │   ├──
```

```
assessment.service.ts
│   │   │   ├── invoice.service.ts
│   │   │   └── file.service.ts
│   │   └── types/
│   │       └── database.types.ts
# GERADO pelo Supabase CLI
│   ├── hooks/
│   │   ├── useStudents.ts
│   │   ├── useWorkouts.ts
│   │   └── useInvoices.ts
│   └── middleware.ts
├── .env.local
└── package.json
```

1.3 Estrutura — fitcoach-pro-mobile

```
fitcoach-pro-mobile/
├── src/
│   ├── app/                          #
Expo Router (file-based)
│   │   ├── (auth)/
│   │   │   ├── login.tsx
│   │   │   └── change-password.tsx
│   │   └── (student)/
│   │       ├── _layout.tsx
# Bottom tab navigator
│   │       ├── index.tsx
# Home
```

```
│   │       ├── workouts/
│   │       │   ├── index.tsx
# Lista de treinos
│   │       │   ├── [id]/
│   │       │   │   ├── preview.tsx
# Modo visualização
│   │       │   │   ├── active.tsx
# Modo execução
│   │       │   │   ├──
evolution.tsx
│   │       │   │   └──
feedbacks.tsx
│   │       ├──
assessments/index.tsx
│   │       ├── progress/index.tsx
│   │       ├── invoices/index.tsx
│   │       ├── files/index.tsx
│   │       └── menu/index.tsx
│   ├── components/
│   │   ├── common/
│   │   │   ├── Header.tsx
│   │   │   ├── BottomNav.tsx
│   │   │   └── VideoPlayer.tsx
│   │   ├── workout/
│   │   │   ├── ExerciseCard.tsx
│   │   │   ├── WorkoutCard.tsx
│   │   │   ├── SeriesTracker.tsx
│   │   │   └── RestTimer.tsx
│   │   └── home/
```

```
│   │       └── WeeklyFrequency.tsx
│   ├── lib/
│   │   ├── supabase.ts
│   │   ├── services/
│   │   │   ├── workout.service.ts
│   │   │   ├── session.service.ts
│   │   │   └── progress.service.ts
│   │   └── types/
│   │       └── database.types.ts
# MESMO arquivo gerado (shared)
│   ├── hooks/
│   │   ├── useAuth.ts
│   │   ├── useWorkouts.ts
│   │   └── useSession.ts
│   └── store/
│       └── workout-session.store.ts
# Zustand — estado da sessão ativa
├── app.json
└── package.json
```

---

2. SCHEMA COMPLETO DO BANCO DE DADOS

> **SDD Rule:** Toda migration deve ter `up` e ser idempotente. Use `IF NOT EXISTS`.

Migration 001 — personals

```
--
20260617000001_create_personals.sql

CREATE EXTENSION IF NOT EXISTS "uuid-
ossp";

CREATE TABLE IF NOT EXISTS
public.personals (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  user_id           UUID          NOT
NULL REFERENCES auth.users(id) ON
DELETE CASCADE,
  name              TEXT          NOT
NULL,
  email             TEXT          NOT
NULL,
  cref              TEXT,
  phone             TEXT,
  instagram_url     TEXT,
  whatsapp_number   TEXT,
  -- White-label
  app_name          TEXT
DEFAULT 'Meu App Fit',
  primary_color     TEXT
DEFAULT '#1B6EE5',
```

```
  logo_url          TEXT,
  splash_url        TEXT,
  welcome_message   TEXT
DEFAULT 'Bem-vindo ao seu treino!',
  -- Plano SaaS
  plan              TEXT          NOT
NULL DEFAULT 'free'

CHECK (plan IN ('free', 'pro',
'premium')),
  plan_expires_at   TIMESTAMPTZ,
  -- Controle
  is_active         BOOLEAN       NOT
NULL DEFAULT true,
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);

-- Índices
CREATE UNIQUE INDEX IF NOT EXISTS
personals_user_id_idx ON
public.personals(user_id);
CREATE UNIQUE INDEX IF NOT EXISTS
personals_email_idx ON
public.personals(email);

-- Trigger: updated_at automático
```

```
CREATE OR REPLACE FUNCTION
public.set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS
$$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$;

CREATE TRIGGER personals_updated_at
  BEFORE UPDATE ON public.personals
  FOR EACH ROW EXECUTE FUNCTION
public.set_updated_at();
```

Migration 002 — students

```
-- 20260617000002_create_students.sql

CREATE TABLE IF NOT EXISTS
public.students (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  user_id           UUID
REFERENCES auth.users(id) ON DELETE
SET NULL,
  personal_id       UUID          NOT
```

```
NULL REFERENCES public.personals(id)
ON DELETE CASCADE,
  name              TEXT          NOT
NULL,
  email             TEXT          NOT
NULL,
  phone             TEXT,
  birth_date        DATE,
  goal              TEXT
CHECK (goal IN (

'lose_weight', 'gain_muscle',

'conditioning', 'health', 'other'
                                    )),
  medical_notes     TEXT,
  photo_url         TEXT,
  -- Plano financeiro
  plan_value        DECIMAL(10,2),
  plan_due_day      SMALLINT
CHECK (plan_due_day BETWEEN 1 AND
31),
  plan_expires_at   DATE,
  -- Dias de treino
[0=Dom,1=Seg,...,6=Sab]
  training_days     SMALLINT[]
DEFAULT '{}',
  -- Controle
  is_active         BOOLEAN       NOT
```

```
NULL DEFAULT true,
  invite_sent_at    TIMESTAMPTZ,
  first_login_at    TIMESTAMPTZ,
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
students_personal_id_idx ON
public.students(personal_id);
CREATE INDEX IF NOT EXISTS
students_user_id_idx ON
public.students(user_id);
CREATE UNIQUE INDEX IF NOT EXISTS
students_email_personal_idx ON
public.students(email, personal_id);

CREATE TRIGGER students_updated_at
  BEFORE UPDATE ON public.students
  FOR EACH ROW EXECUTE FUNCTION
public.set_updated_at();
```

Migration 003 — exercises

```
--
20260617000003_create_exercises.sql
```

```
CREATE TYPE public.muscle_group AS
ENUM (
  'chest', 'back', 'legs',
'shoulders',
  'biceps', 'triceps', 'abs',
'cardio', 'functional', 'other'
);

CREATE TABLE IF NOT EXISTS
public.exercises (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  personal_id       UUID          NOT
NULL REFERENCES public.personals(id)
ON DELETE CASCADE,
  name              TEXT          NOT
NULL,
  muscle_group
public.muscle_group NOT NULL DEFAULT
'other',
  instructions      TEXT,
  video_url         TEXT,           -
- URL YouTube, Instagram ou Supabase
Storage
  video_type        TEXT
CHECK (video_type IN ('youtube',
'instagram', 'upload')),
```

```
  thumbnail_url     TEXT,
  is_archived       BOOLEAN       NOT
NULL DEFAULT false,
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
exercises_personal_id_idx ON
public.exercises(personal_id);
CREATE INDEX IF NOT EXISTS
exercises_muscle_group_idx ON
public.exercises(muscle_group);

CREATE TRIGGER exercises_updated_at
  BEFORE UPDATE ON public.exercises
  FOR EACH ROW EXECUTE FUNCTION
public.set_updated_at();
```

Migration 004 — workout_plans e workout_exercises

```
--
20260617000004_create_workout_plans.s
ql
```

```
CREATE TABLE IF NOT EXISTS
public.workout_plans (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  personal_id       UUID          NOT
NULL REFERENCES public.personals(id)
ON DELETE CASCADE,
  student_id        UUID          NOT
NULL REFERENCES public.students(id)
ON DELETE CASCADE,
  name              TEXT          NOT
NULL,           -- "Treino 1",
"Treino A"
  description       TEXT,
-- "Peito / Tríceps / Ombros"
  is_active         BOOLEAN       NOT
NULL DEFAULT true,
  is_extra          BOOLEAN       NOT
NULL DEFAULT false,
  is_published      BOOLEAN       NOT
NULL DEFAULT false,
  display_order     SMALLINT      NOT
NULL DEFAULT 0,
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);
```

```
CREATE INDEX IF NOT EXISTS
workout_plans_student_id_idx ON
public.workout_plans(student_id);
CREATE INDEX IF NOT EXISTS
workout_plans_personal_id_idx ON
public.workout_plans(personal_id);

CREATE TRIGGER
workout_plans_updated_at
  BEFORE UPDATE ON
public.workout_plans
  FOR EACH ROW EXECUTE FUNCTION
public.set_updated_at();

-- Exercícios dentro de um plano de
treino
CREATE TABLE IF NOT EXISTS
public.workout_exercises (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  plan_id           UUID          NOT
NULL REFERENCES
public.workout_plans(id) ON DELETE
CASCADE,
  exercise_id       UUID          NOT
NULL REFERENCES public.exercises(id)
ON DELETE RESTRICT,
```

```
  sets              TEXT          NOT
NULL DEFAULT '3',    -- "4", "3"
  reps              TEXT          NOT
NULL DEFAULT '12',   -- "10 a 12",
"30s", "até a falha"
  load              TEXT,
-- "10/10", "0kg", "corporal"
  rest_seconds      SMALLINT
DEFAULT 60,
  instructions      TEXT,
  display_order     SMALLINT      NOT
NULL DEFAULT 0,
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
workout_exercises_plan_id_idx ON
public.workout_exercises(plan_id);
```

Migration 005 — workout_sessions e session_exercise_logs

```
--
20260617000005_create_workout_session
s.sql

CREATE TABLE IF NOT EXISTS
```

```
public.workout_sessions (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  plan_id           UUID          NOT
NULL REFERENCES
public.workout_plans(id) ON DELETE
CASCADE,
  student_id        UUID          NOT
NULL REFERENCES public.students(id)
ON DELETE CASCADE,
  -- Tempo
  started_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW(),
  finished_at       TIMESTAMPTZ,
  duration_seconds  INT
GENERATED ALWAYS AS (
                      EXTRACT(EPOCH
FROM (finished_at - started_at))::INT
                    ) STORED,
  -- Feedback
  feedback_rating   SMALLINT
CHECK (feedback_rating BETWEEN 1 AND
5),
  feedback_notes    TEXT,
  -- Status
  is_completed      BOOLEAN       NOT
NULL DEFAULT false,
  created_at        TIMESTAMPTZ   NOT
```

```
NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
sessions_student_id_idx ON
public.workout_sessions(student_id);
CREATE INDEX IF NOT EXISTS
sessions_plan_id_idx ON
public.workout_sessions(plan_id);
CREATE INDEX IF NOT EXISTS
sessions_started_at_idx ON
public.workout_sessions(started_at
DESC);

-- Log de cargas por série
CREATE TABLE IF NOT EXISTS
public.session_exercise_logs (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  session_id        UUID          NOT
NULL REFERENCES
public.workout_sessions(id) ON DELETE
CASCADE,
  workout_exercise_id UUID        NOT
NULL REFERENCES
public.workout_exercises(id) ON
DELETE CASCADE,
  set_number        SMALLINT      NOT
```

```
NULL,
  load_used         TEXT,
  reps_done         TEXT,
  is_completed      BOOLEAN       NOT
NULL DEFAULT false,
  logged_at         TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
logs_session_id_idx ON
public.session_exercise_logs(session_
id);
```

Migration 006 — physical_assessments

```
--
20260617000006_create_assessments.sql

CREATE TABLE IF NOT EXISTS
public.physical_assessments (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  student_id        UUID          NOT
NULL REFERENCES public.students(id)
ON DELETE CASCADE,
  personal_id       UUID          NOT
```

```
NULL REFERENCES public.personals(id)
ON DELETE CASCADE,
  assessed_at       DATE          NOT
NULL DEFAULT CURRENT_DATE,
  -- Composição corporal
  weight_kg         DECIMAL(5,2),
  height_cm         DECIMAL(5,2),
  bmi               DECIMAL(5,2)
GENERATED ALWAYS AS (
                      ROUND(weight_kg
/ POWER(height_cm / 100.0, 2), 2)
                    ) STORED,
  body_fat_pct      DECIMAL(5,2),
  lean_mass_kg      DECIMAL(5,2),
  fat_mass_kg       DECIMAL(5,2),
  -- Medidas em cm
  neck_cm           DECIMAL(5,2),
  shoulder_cm       DECIMAL(5,2),
  chest_cm          DECIMAL(5,2),
  waist_cm          DECIMAL(5,2),
  abdomen_cm        DECIMAL(5,2),
  hip_cm            DECIMAL(5,2),
  thigh_left_cm     DECIMAL(5,2),
  thigh_right_cm    DECIMAL(5,2),
  calf_left_cm      DECIMAL(5,2),
  calf_right_cm     DECIMAL(5,2),
  bicep_left_cm     DECIMAL(5,2),
  bicep_right_cm    DECIMAL(5,2),
  -- Dobras cutâneas (mm) — Protocolo
```

```
Pollock 7 dobras
  skinfold_chest    DECIMAL(5,2),
  skinfold_midaxil  DECIMAL(5,2),
  skinfold_triceps  DECIMAL(5,2),
  skinfold_subscap  DECIMAL(5,2),
  skinfold_abdomen  DECIMAL(5,2),
  skinfold_suprail  DECIMAL(5,2),
  skinfold_thigh    DECIMAL(5,2),
  -- Fotos
  photo_front_url   TEXT,
  photo_side_url    TEXT,
  photo_back_url    TEXT,
  -- Laudo
  notes             TEXT,
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
assessments_student_id_idx ON
public.physical_assessments(student_i
d);
CREATE INDEX IF NOT EXISTS
assessments_assessed_at_idx ON
public.physical_assessments(assessed_
at DESC);
```

```
CREATE TRIGGER assessments_updated_at
  BEFORE UPDATE ON
public.physical_assessments
  FOR EACH ROW EXECUTE FUNCTION
public.set_updated_at();

-- Progresso auto-registrado pelo
aluno
CREATE TABLE IF NOT EXISTS
public.student_progress (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  student_id        UUID          NOT
NULL REFERENCES public.students(id)
ON DELETE CASCADE,
  recorded_at       DATE          NOT
NULL DEFAULT CURRENT_DATE,
  weight_kg         DECIMAL(5,2),
  photo_url         TEXT,
  notes             TEXT,
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
progress_student_id_idx ON
public.student_progress(student_id);
```

Migration 007 — invoices

```
-- 20260617000007_create_invoices.sql

CREATE TYPE public.invoice_status AS
ENUM (
  'pending', 'paid', 'overdue',
'cancelled'
);

CREATE TABLE IF NOT EXISTS
public.invoices (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  student_id        UUID
NOT NULL REFERENCES
public.students(id) ON DELETE
CASCADE,
  personal_id       UUID
NOT NULL REFERENCES
public.personals(id) ON DELETE
CASCADE,
  amount            DECIMAL(10,2)
NOT NULL,
  due_date          DATE
NOT NULL,
  paid_at           TIMESTAMPTZ,
  status
```

```
public.invoice_status NOT NULL
DEFAULT 'pending',
  payment_method    TEXT
CHECK (payment_method IN ('pix',
'boleto', 'credit_card', 'cash',
'other')),
  payment_link      TEXT,
  pix_key           TEXT,
  reference_month   TEXT,           -
- "2026-06" (AAAA-MM)
  notes             TEXT,
  created_at        TIMESTAMPTZ
NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ
NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
invoices_student_id_idx ON
public.invoices(student_id);
CREATE INDEX IF NOT EXISTS
invoices_personal_id_idx ON
public.invoices(personal_id);
CREATE INDEX IF NOT EXISTS
invoices_status_idx ON
public.invoices(status);
CREATE INDEX IF NOT EXISTS
invoices_due_date_idx ON
public.invoices(due_date);
```

```
CREATE TRIGGER invoices_updated_at
  BEFORE UPDATE ON public.invoices
  FOR EACH ROW EXECUTE FUNCTION
public.set_updated_at();
```

Migration 008 — files

```
-- 20260617000008_create_files.sql

CREATE TABLE IF NOT EXISTS
public.files (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  personal_id       UUID          NOT
NULL REFERENCES public.personals(id)
ON DELETE CASCADE,
  student_id        UUID
REFERENCES public.students(id) ON
DELETE CASCADE,
                    -- NULL =
disponível para todos os alunos do
personal
  name              TEXT          NOT
NULL,
  description       TEXT,
  file_url          TEXT          NOT
```

```
NULL,
  file_type         TEXT          NOT
NULL CHECK (file_type IN ('pdf',
'image', 'video', 'other')),
  file_size_bytes   BIGINT,
  category          TEXT,           -
- "dieta", "protocolo", "laudo",
"outros"
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
files_personal_id_idx ON
public.files(personal_id);
CREATE INDEX IF NOT EXISTS
files_student_id_idx ON
public.files(student_id);
```

Migration 009 — notifications

```
--
20260617000009_create_notifications.s
ql

CREATE TYPE
public.notification_recipient AS ENUM
('student', 'personal');
```

```
CREATE TABLE IF NOT EXISTS
public.notifications (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  recipient_id      UUID
NOT NULL,
  recipient_type
public.notification_recipient NOT
NULL,
  title             TEXT
NOT NULL,
  body              TEXT
NOT NULL,
  type              TEXT
NOT NULL, -- 'new_workout',
'invoice_due', 'feedback', etc.
  data              JSONB
DEFAULT '{}',
  read_at           TIMESTAMPTZ,
  push_sent_at      TIMESTAMPTZ,
  created_at        TIMESTAMPTZ
NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS
notif_recipient_id_idx ON
public.notifications(recipient_id);
```

```
CREATE INDEX IF NOT EXISTS
notif_read_at_idx ON
public.notifications(read_at) WHERE
read_at IS NULL;
CREATE INDEX IF NOT EXISTS
notif_created_at_idx ON
public.notifications(created_at
DESC);

-- Push Tokens dos dispositivos
CREATE TABLE IF NOT EXISTS
public.push_tokens (
  id                UUID
PRIMARY KEY DEFAULT
uuid_generate_v4(),
  user_id           UUID          NOT
NULL REFERENCES auth.users(id) ON
DELETE CASCADE,
  token             TEXT          NOT
NULL,
  platform          TEXT          NOT
NULL CHECK (platform IN ('android',
'ios', 'web')),
  created_at        TIMESTAMPTZ   NOT
NULL DEFAULT NOW(),
  UNIQUE(user_id, token)
);
```

Migration 011 — Functions e Triggers de Negócio

```
--
20260617000011_functions_triggers.sql

-- 1. Criar perfil do personal
automaticamente após signup
CREATE OR REPLACE FUNCTION
public.handle_new_personal()
RETURNS TRIGGER LANGUAGE plpgsql
SECURITY DEFINER AS $$
BEGIN
  INSERT INTO public.personals
(user_id, name, email)
  VALUES (
    NEW.id,
    COALESCE(NEW.raw_user_meta_data-
>>'name', 'Personal'),
    NEW.email
  );
  RETURN NEW;
END;
$$;

-- Este trigger só dispara para
personals (role='personal' no
metadata)
```

```
CREATE OR REPLACE TRIGGER
on_personal_signup
  AFTER INSERT ON auth.users
  FOR EACH ROW
  WHEN (NEW.raw_user_meta_data-
>>'role' = 'personal')
  EXECUTE FUNCTION
public.handle_new_personal();

-- 2. Vincular aluno ao auth.user
após aceitar convite
CREATE OR REPLACE FUNCTION
public.link_student_to_user()
RETURNS TRIGGER LANGUAGE plpgsql
SECURITY DEFINER AS $$
BEGIN
  -- Quando aluno fizer primeiro
login, vincular student ao user_id
  UPDATE public.students
  SET user_id = NEW.id,
first_login_at = NOW()
  WHERE email = NEW.email AND user_id
IS NULL;
  RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER
on_student_signup
```

```
  AFTER INSERT ON auth.users
  FOR EACH ROW
  WHEN (NEW.raw_user_meta_data-
>>'role' = 'student')
  EXECUTE FUNCTION
public.link_student_to_user();

-- 3. Contagem de execuções do treino
(VIEW materializada)
CREATE OR REPLACE VIEW
public.workout_plan_stats AS
SELECT
  wp.id AS plan_id,
  wp.student_id,
  wp.name,
  COUNT(ws.id) AS total_sessions,
  MAX(ws.started_at) AS
last_session_at
FROM public.workout_plans wp
LEFT JOIN public.workout_sessions ws
  ON ws.plan_id = wp.id AND
ws.is_completed = true
GROUP BY wp.id, wp.student_id,
wp.name;

-- 4. Frequência semanal do aluno
(VIEW)
CREATE OR REPLACE VIEW
public.student_weekly_frequency AS
```

```
SELECT
  s.id AS student_id,
  s.personal_id,
  EXTRACT(ISODOW FROM
ws.started_at)::INT AS day_of_week,
  DATE_TRUNC('week', ws.started_at)
AS week_start,
  COUNT(*) AS sessions_count
FROM public.students s
JOIN public.workout_sessions ws ON
ws.student_id = s.id AND
ws.is_completed = true
GROUP BY s.id, s.personal_id,
day_of_week, week_start;
```

---

3. ROW LEVEL SECURITY (RLS)

> **SDD Rule:** RLS é ativado em TODAS as tabelas. Nenhuma tabela fica sem política.

```
-- 20260617000010_rls_policies.sql

-- Helper functions
CREATE OR REPLACE FUNCTION
public.get_personal_id()
RETURNS UUID LANGUAGE sql STABLE AS
$$
```

```
  SELECT id FROM public.personals
WHERE user_id = auth.uid()
$$;

CREATE OR REPLACE FUNCTION
public.get_student_id()
RETURNS UUID LANGUAGE sql STABLE AS
$$
  SELECT id FROM public.students
WHERE user_id = auth.uid()
$$;

CREATE OR REPLACE FUNCTION
public.is_personal()
RETURNS BOOLEAN LANGUAGE sql STABLE
AS $$
  SELECT EXISTS (SELECT 1 FROM
public.personals WHERE user_id =
auth.uid())
$$;

--
=====================================
======================
-- TABELA: personals
--
=====================================
======================
ALTER TABLE public.personals ENABLE
```

```
ROW LEVEL SECURITY;

CREATE POLICY "personal_select_own"
  ON public.personals FOR SELECT
  USING (user_id = auth.uid());

CREATE POLICY "personal_update_own"
  ON public.personals FOR UPDATE
  USING (user_id = auth.uid());

--
========================================
=======================
-- TABELA: students
--
========================================
=======================
ALTER TABLE public.students ENABLE
ROW LEVEL SECURITY;

-- Personal vê seus próprios alunos
CREATE POLICY
"personal_manages_students"
  ON public.students FOR ALL
  USING (personal_id =
public.get_personal_id());

-- Aluno vê apenas seus próprios
dados
```

```
CREATE POLICY "student_sees_own"
  ON public.students FOR SELECT
  USING (user_id = auth.uid());

CREATE POLICY "student_updates_own"
  ON public.students FOR UPDATE
  USING (user_id = auth.uid())
  WITH CHECK (user_id = auth.uid());

--
=======================================
=======================
-- TABELA: exercises
--
=======================================
=======================
ALTER TABLE public.exercises ENABLE
ROW LEVEL SECURITY;

CREATE POLICY
"personal_manages_exercises"
  ON public.exercises FOR ALL
  USING (personal_id =
public.get_personal_id());

-- Aluno vê exercícios dos planos que
lhe foram atribuídos
CREATE POLICY
"student_sees_exercises"
```

```
  ON public.exercises FOR SELECT
  USING (
    personal_id IN (
      SELECT personal_id FROM
public.students WHERE user_id =
auth.uid()
    )
  );

--
=====================================
======================
-- TABELA: workout_plans
--
=====================================
======================
ALTER TABLE public.workout_plans
ENABLE ROW LEVEL SECURITY;

CREATE POLICY
"personal_manages_plans"
  ON public.workout_plans FOR ALL
  USING (personal_id =
public.get_personal_id());

CREATE POLICY
"student_sees_own_plans"
  ON public.workout_plans FOR SELECT
  USING (
```

```
    student_id =
public.get_student_id()
    AND is_published = true
  );

--
====================================
======================
-- TABELA: workout_exercises
--
====================================
======================
ALTER TABLE public.workout_exercises
ENABLE ROW LEVEL SECURITY;

CREATE POLICY
"personal_manages_workout_exercises"
  ON public.workout_exercises FOR ALL
  USING (
    plan_id IN (
      SELECT id FROM
public.workout_plans
      WHERE personal_id =
public.get_personal_id()
    )
  );

CREATE POLICY
"student_sees_workout_exercises"
```

```
  ON public.workout_exercises FOR
SELECT
  USING (
    plan_id IN (
      SELECT id FROM
public.workout_plans
      WHERE student_id =
public.get_student_id() AND
is_published = true
    )
  );

--
=====================================
=======================
-- TABELA: workout_sessions
--
=====================================
=======================
ALTER TABLE public.workout_sessions
ENABLE ROW LEVEL SECURITY;

-- Personal vê sessões dos seus
alunos
CREATE POLICY
"personal_sees_sessions"
  ON public.workout_sessions FOR
SELECT
  USING (
```

```
    student_id IN (
      SELECT id FROM public.students
WHERE personal_id =
public.get_personal_id()
    )
  );

-- Aluno gerencia suas próprias
sessões
CREATE POLICY
"student_manages_sessions"
  ON public.workout_sessions FOR ALL
  USING (student_id =
public.get_student_id());

--
=====================================
======================
-- TABELA: session_exercise_logs
--
=====================================
======================
ALTER TABLE
public.session_exercise_logs ENABLE
ROW LEVEL SECURITY;

CREATE POLICY "student_manages_logs"
  ON public.session_exercise_logs FOR
ALL
```

```
  USING (
    session_id IN (
      SELECT id FROM
public.workout_sessions
      WHERE student_id =
public.get_student_id()
    )
  );

CREATE POLICY "personal_sees_logs"
  ON public.session_exercise_logs FOR
SELECT
  USING (
    session_id IN (
      SELECT ws.id FROM
public.workout_sessions ws
      JOIN public.students s ON s.id
= ws.student_id
      WHERE s.personal_id =
public.get_personal_id()
    )
  );

--
=====================================
======================
-- TABELA: physical_assessments
--
=====================================
```

```
======================
ALTER TABLE
public.physical_assessments ENABLE
ROW LEVEL SECURITY;

CREATE POLICY
"personal_manages_assessments"
  ON public.physical_assessments FOR
ALL
  USING (personal_id =
public.get_personal_id());

CREATE POLICY
"student_sees_own_assessments"
  ON public.physical_assessments FOR
SELECT
  USING (student_id =
public.get_student_id());

--
====================================
======================
-- TABELA: student_progress
--
====================================
======================
ALTER TABLE public.student_progress
ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY
"student_manages_progress"
  ON public.student_progress FOR ALL
  USING (student_id =
public.get_student_id());

CREATE POLICY
"personal_sees_student_progress"
  ON public.student_progress FOR
SELECT
  USING (
    student_id IN (
      SELECT id FROM public.students
WHERE personal_id =
public.get_personal_id()
    )
  );

--
=====================================
======================
-- TABELA: invoices
--
=====================================
======================
ALTER TABLE public.invoices ENABLE
ROW LEVEL SECURITY;

CREATE POLICY
```

```
"personal_manages_invoices"
  ON public.invoices FOR ALL
  USING (personal_id =
public.get_personal_id());

CREATE POLICY
"student_sees_own_invoices"
  ON public.invoices FOR SELECT
  USING (student_id =
public.get_student_id());

--
=====================================
======================
-- TABELA: files
--
=====================================
======================
ALTER TABLE public.files ENABLE ROW
LEVEL SECURITY;

CREATE POLICY
"personal_manages_files"
  ON public.files FOR ALL
  USING (personal_id =
public.get_personal_id());

-- Aluno vê arquivos globais do
personal OU específicos dele
```

```
CREATE POLICY "student_sees_files"
  ON public.files FOR SELECT
  USING (
    personal_id IN (
      SELECT personal_id FROM
public.students WHERE user_id =
auth.uid()
    )
    AND (
      student_id IS NULL OR
student_id = public.get_student_id()
    )
  );

--
=====================================
======================
-- TABELA: notifications
--
=====================================
======================
ALTER TABLE public.notifications
ENABLE ROW LEVEL SECURITY;

CREATE POLICY
"user_sees_own_notifications"
  ON public.notifications FOR SELECT
  USING (recipient_id = auth.uid()
    OR recipient_id IN (
```

```
      SELECT id FROM public.students
WHERE user_id = auth.uid()
    )
    OR recipient_id IN (
      SELECT id FROM public.personals
WHERE user_id = auth.uid()
    )
  );

CREATE POLICY
"user_updates_own_notifications"
  ON public.notifications FOR UPDATE
  USING (recipient_id = auth.uid());
```

---

4. STORAGE BUCKETS

```
-- Configurar via Supabase Dashboard
ou CLI

-- Bucket: exercise-videos (público
para leitura)
INSERT INTO storage.buckets (id,
name, public, file_size_limit,
allowed_mime_types)
VALUES (
  'exercise-videos', 'exercise-
videos', true,
```

```
  104857600,  -- 100MB
  ARRAY['video/mp4',
'video/quicktime', 'video/webm']
);

-- Bucket: exercise-thumbnails
(público)
INSERT INTO storage.buckets (id,
name, public, file_size_limit,
allowed_mime_types)
VALUES (
  'exercise-thumbnails', 'exercise-
thumbnails', true,
  5242880,   -- 5MB
  ARRAY['image/jpeg', 'image/png',
'image/webp']
);

-- Bucket: assessment-photos (privado
— acesso via signed URL)
INSERT INTO storage.buckets (id,
name, public, file_size_limit,
allowed_mime_types)
VALUES (
  'assessment-photos', 'assessment-
photos', false,
  10485760,  -- 10MB
  ARRAY['image/jpeg', 'image/png',
'image/webp']
```

```
);

-- Bucket: student-files (privado)
INSERT INTO storage.buckets (id,
name, public, file_size_limit,
allowed_mime_types)
VALUES (
  'student-files', 'student-files',
false,
  52428800,  -- 50MB
  ARRAY['application/pdf',
'image/jpeg', 'image/png']
);

-- Bucket: avatars (público)
INSERT INTO storage.buckets (id,
name, public, file_size_limit,
allowed_mime_types)
VALUES (
  'avatars', 'avatars', true,
  2097152,   -- 2MB
  ARRAY['image/jpeg', 'image/png',
'image/webp']
);
```

Políticas de Storage:

```
-- Personal gerencia seus próprios
arquivos
```

```
-- Caminho padrão:
{personal_id}/{filename}

CREATE POLICY
"personal_upload_exercises"
  ON storage.objects FOR INSERT
  WITH CHECK (
    bucket_id = 'exercise-videos'
    AND (storage.foldername(name))[1]
= public.get_personal_id()::text
  );

-- Aluno acessa fotos de avaliação
apenas do seu personal
CREATE POLICY
"student_view_assessment_photos"
  ON storage.objects FOR SELECT
  USING (
    bucket_id = 'assessment-photos'
    AND (storage.foldername(name))[1]
IN (
      SELECT personal_id::text FROM
public.students WHERE user_id =
auth.uid()
    )
  );
```

5. SUPABASE AUTH

Configuração

```
# supabase/config.toml

[auth]
site_url =
"https://fitcoachpro.vercel.app"
additional_redirect_urls = [
  "exp://localhost:8081",          #
Expo dev
  "fitcoachpro://",                #
Deep link mobile
]
jwt_expiry = 3600
enable_refresh_token_rotation = true

[auth.email]
enable_signup = true
double_confirm_changes = true
enable_confirmations = true
```

Roles e Metadata

```
// Personal Trainer — signup
await supabase.auth.signUp({
```

```
  email,
  password,
  options: {
    data: {
      role: 'personal',
      name: personalName
    }
  }
})

// Aluno — convidado via Edge
Function
// O personal chama invite-student
que cria o user com:
{
  role: 'student',
  personal_id: 'uuid-do-personal',
  student_id: 'uuid-do-student-
record'
}
```

---

6. EDGE FUNCTIONS

6.1 invite-student

```
// supabase/functions/invite-
student/index.ts
// Chamada pelo personal para criar
conta do aluno

import { serve } from
"https://deno.land/std@0.177.0/http/s
erver.ts"
import { createClient } from
"https://esm.sh/@supabase/supabase-
js@2"

serve(async (req) => {
  const { student_id } = await
req.json()

  const supabaseAdmin = createClient(
    Deno.env.get('SUPABASE_URL')!,

Deno.env.get('SUPABASE_SERVICE_ROLE_K
EY')!
  )

  // Buscar dados do aluno
  const { data: student } = await
supabaseAdmin
    .from('students')
    .select('*, personals(app_name,
primary_color)')
```

```
    .eq('id', student_id)
    .single()

  // Criar usuário auth com senha
temporária
  const tempPassword =
Math.random().toString(36).slice(-8).
toUpperCase()

  const { data: authUser } = await
supabaseAdmin.auth.admin.createUser({
    email: student.email,
    password: tempPassword,
    email_confirm: true,
    user_metadata: {
      role: 'student',
      name: student.name,
      personal_id:
student.personal_id,
      student_id: student.id,
      must_change_password: true
    }
  })

  // Atualizar student com user_id e
invite_sent_at
  await supabaseAdmin
    .from('students')
    .update({
```

```
      user_id: authUser.user!.id,
      invite_sent_at: new
Date().toISOString()
    })
    .eq('id', student_id)

  // TODO: Enviar e-mail com
credenciais via Resend

  return new
Response(JSON.stringify({
    success: true,
    temp_password: tempPassword
  }))
})
```

6.2 overdue-invoices (Cron Job)

```
// supabase/functions/overdue-
invoices/index.ts
// Executado diariamente: atualiza
faturas vencidas

serve(async (_req) => {
  const supabase = createClient(/*
service role */)

  const { data, error } = await
```

```
supabase
    .from('invoices')
    .update({ status: 'overdue' })
    .eq('status', 'pending')
    .lt('due_date', new
Date().toISOString().split('T')[0])
    .select('id, student_id, amount')

  // Criar notificações para os
alunos com faturas vencidas
  if (data && data.length > 0) {
    const notifications =
data.map(inv => ({
      recipient_id: inv.student_id,
      recipient_type: 'student',
      title: 'Fatura vencida',
      body: `Sua fatura de R$
${inv.amount} está vencida.`,
      type: 'invoice_overdue',
      data: { invoice_id: inv.id }
    }))

    await
supabase.from('notifications').insert
(notifications)
  }

  return new
Response(JSON.stringify({ updated:
```

```
data?.length ?? 0 }))
})
```

---

7. TIPOS TYPESCRIPT GERADOS

```
# Comando para gerar — executar após cada migration
npx supabase gen types typescript \
  --project-id SEU_PROJECT_ID \
  --schema public \
  > src/lib/types/database.types.ts
```

```
// src/lib/types/database.types.ts (exemplo parcial gerado)

export type Database = {
  public: {
    Tables: {
      personals: {
        Row: {
          id: string
          user_id: string
          name: string
          email: string
```

```
          cref: string | null
          phone: string | null
          app_name: string
          primary_color: string
          logo_url: string | null
          plan: 'free' | 'pro' |
'premium'
          plan_expires_at: string |
null
          is_active: boolean
          created_at: string
          updated_at: string
        }
        Insert: Omit<Row, 'id' |
'created_at' | 'updated_at'>
        Update: Partial<Insert>
      }
      students: {
        Row: {
          id: string
          user_id: string | null
          personal_id: string
          name: string
          email: string
          phone: string | null
          birth_date: string | null
          goal: 'lose_weight' |
'gain_muscle' | 'conditioning' |
'health' | 'other' | null
```

```
          plan_value: number | null
          is_active: boolean
          training_days: number[]
          created_at: string
          updated_at: string
        }
        Insert: Omit<Row, 'id' |
'created_at' | 'updated_at'>
        Update: Partial<Insert>
      }
      // ... demais tabelas
    }
    Views: {
      workout_plan_stats: {
        Row: {
          plan_id: string
          student_id: string
          name: string
          total_sessions: number
          last_session_at: string |
null
        }
      }
    }
    Enums: {
      muscle_group: 'chest' | 'back'
| 'legs' | 'shoulders' | 'biceps' |
'triceps' | 'abs' | 'cardio' |
'functional' | 'other'
```

```
      invoice_status: 'pending' |
'paid' | 'overdue' | 'cancelled'
    }
  }
}

// Aliases convenientes
export type Tables<T extends keyof
Database['public']['Tables']> =
  Database['public']['Tables'][T]
['Row']

export type Personal =
Tables<'personals'>
export type Student =
Tables<'students'>
export type Exercise =
Tables<'exercises'>
export type WorkoutPlan =
Tables<'workout_plans'>
export type WorkoutExercise =
Tables<'workout_exercises'>
export type WorkoutSession =
Tables<'workout_sessions'>
export type Invoice =
Tables<'invoices'>
```

---

8. SERVICE LAYER

> **SDD Rule:** Toda operação de banco passa pelo Service Layer. Zero queries diretas em componentes.

8.1 student.service.ts

```
//
src/lib/services/student.service.ts

import { createServerClient } from
'@/lib/supabase/server'
import type { Student, Database }
from '@/lib/types/database.types'

type StudentInsert =
Database['public']['Tables']
['students']['Insert']
type StudentUpdate =
Database['public']['Tables']
['students']['Update']

export const StudentService = {

  async list(personalId: string) {
    const supabase =
createServerClient()
```

```
    return supabase
      .from('students')
      .select('*')
      .eq('personal_id', personalId)
      .order('name')
  },

  async getById(id: string) {
    const supabase =
createServerClient()
    return supabase
      .from('students')
      .select(`
        *,
        workout_plans (
          id, name, description,
is_active, is_published,
display_order,
          workout_plan_stats
(total_sessions, last_session_at)
        ),
        invoices (id, amount,
due_date, status)
      `)
      .eq('id', id)
      .single()
  },

  async create(data: StudentInsert) {
```

```
    const supabase =
createServerClient()
    return supabase
      .from('students')
      .insert(data)
      .select()
      .single()
  },

  async update(id: string, data:
StudentUpdate) {
    const supabase =
createServerClient()
    return supabase
      .from('students')
      .update(data)
      .eq('id', id)
      .select()
      .single()
  },

  async deactivate(id: string) {
    return StudentService.update(id,
{ is_active: false })
  },

  async getFrequency(studentId:
string, weeks = 4) {
    const supabase =
```

```
createServerClient()
    const since = new Date()
    since.setDate(since.getDate() -
weeks * 7)

    return supabase
      .from('workout_sessions')
      .select('started_at, plan_id')
      .eq('student_id', studentId)
      .eq('is_completed', true)
      .gte('started_at',
since.toISOString())
      .order('started_at', {
ascending: false })
  }
}
```

8.2 workout.service.ts

```
//
src/lib/services/workout.service.ts

export const WorkoutService = {

  async listByStudent(studentId:
string) {
    const supabase =
createServerClient()
```

```
    return supabase
      .from('workout_plans')
      .select(`
        *,
        workout_plan_stats
(total_sessions, last_session_at),
        workout_exercises (
          id, sets, reps, load,
rest_seconds, instructions,
display_order,
          exercises (id, name,
muscle_group, video_url,
thumbnail_url, instructions)
        )
      `)
      .eq('student_id', studentId)
      .eq('is_active', true)
      .order('display_order')
  },

  async publish(planId: string) {
    const supabase =
createServerClient()
    return supabase
      .from('workout_plans')
      .update({ is_published: true })
      .eq('id', planId)
  },
```

```
  async addExercise(planId: string,
exerciseData: {
    exercise_id: string
    sets: string
    reps: string
    load?: string
    rest_seconds?: number
    instructions?: string
    display_order: number
  }) {
    const supabase =
createServerClient()
    return supabase
      .from('workout_exercises')
      .insert({ plan_id: planId,
...exerciseData })
      .select()
      .single()
  },

  async reorderExercises(exercises: {
id: string; display_order: number }
[]) {
    const supabase =
createServerClient()
    const updates = exercises.map(e
=>
      supabase
        .from('workout_exercises')
```

```
        .update({ display_order:
e.display_order })
        .eq('id', e.id)
    )
    return Promise.all(updates)
  }
}
```

8.3 session.service.ts (Mobile)

```
//
mobile/src/lib/services/session.servi
ce.ts

export const SessionService = {

  async start(planId: string,
studentId: string) {
    const supabase =
getSupabaseClient()
    return supabase
      .from('workout_sessions')
      .insert({
        plan_id: planId,
        student_id: studentId,
        started_at: new
Date().toISOString()
      })
```

```
      .select()
      .single()
  },

  async logSet(sessionId: string,
data: {
    workout_exercise_id: string
    set_number: number
    load_used?: string
    reps_done?: string
    is_completed: boolean
  }) {
    const supabase =
getSupabaseClient()
    return supabase
      .from('session_exercise_logs')
      .upsert({
        session_id: sessionId,
        ...data,
        logged_at: new
Date().toISOString()
      }, {
        onConflict:
'session_id,workout_exercise_id,set_n
umber'
      })
  },

  async finish(sessionId: string,
```

```
feedback?: {
    rating: number
    notes?: string
  }) {
    const supabase =
getSupabaseClient()
    return supabase
      .from('workout_sessions')
      .update({
        finished_at: new
Date().toISOString(),
        is_completed: true,
        feedback_rating:
feedback?.rating,
        feedback_notes:
feedback?.notes
      })
      .eq('id', sessionId)
  }
}
```

---

9. ARQUITETURA NEXT.JS — DASHBOARD WEB

9.1 Supabase Clients

```
// src/lib/supabase/server.ts —
Server Components e Server Actions
import { createServerClient as
_create } from '@supabase/ssr'
import { cookies } from
'next/headers'
import type { Database } from
'@/lib/types/database.types'

export function createServerClient()
{
  const cookieStore = cookies()
  return _create<Database>(

process.env.NEXT_PUBLIC_SUPABASE_URL!
,

process.env.NEXT_PUBLIC_SUPABASE_ANON
_KEY!,
    {
      cookies: {
        get: (name) =>
cookieStore.get(name)?.value,
        set: (name, value, options)
=> cookieStore.set({ name, value,
...options }),
        remove: (name, options) =>
cookieStore.set({ name, value: '',
...options })
```

```
      }
    }
  )
}

// src/lib/supabase/client.ts —
Client Components
import { createBrowserClient } from
'@supabase/ssr'
import type { Database } from
'@/lib/types/database.types'

export const supabase =
createBrowserClient<Database>(

process.env.NEXT_PUBLIC_SUPABASE_URL!
,

process.env.NEXT_PUBLIC_SUPABASE_ANON
_KEY!
)
```

9.2 Middleware de Auth

```
// src/middleware.ts
import { createServerClient } from
'@supabase/ssr'
import { NextResponse } from
```

```
'next/server'
import type { NextRequest } from
'next/server'

export async function
middleware(request: NextRequest) {
  const response =
NextResponse.next()
  const supabase =
createServerClient(/* ... */)

  const { data: { session } } = await
supabase.auth.getSession()

  // Rotas protegidas do dashboard
  if
(request.nextUrl.pathname.startsWith(
'/dashboard')) {
    if (!session) {
      return
NextResponse.redirect(new
URL('/login', request.url))
    }
    // Verificar se é personal (tem
registro em personals)
    const { data: personal } = await
supabase
      .from('personals')
      .select('id')
```

```
      .eq('user_id', session.user.id)
      .single()

    if (!personal) {
      return
NextResponse.redirect(new
URL('/login', request.url))
    }
  }

  return response
}

export const config = {
  matcher: ['/dashboard/:path*']
}
```

9.3 Server Actions

```
//
src/app/(dashboard)/students/actions.
ts
'use server'

import { revalidatePath } from
'next/cache'
import { StudentService } from
'@/lib/services/student.service'
```

```
import { createClient } from
'@supabase/supabase-js'

export async function
createStudentAction(formData:
FormData) {
  const data = {
    name: formData.get('name') as
string,
    email: formData.get('email') as
string,
    phone: formData.get('phone') as
string,
    // ... outros campos
  }

  const { data: student, error } =
await StudentService.create({
    ...data,
    personal_id: await
getPersonalId() // helper que pega do
session
  })

  if (error) throw error

  // Enviar convite via Edge Function
  await
fetch(`${process.env.SUPABASE_URL}/fu
```

```
nctions/v1/invite-student`, {
    method: 'POST',
    headers: {
      Authorization: `Bearer
${process.env.SUPABASE_SERVICE_ROLE_K
EY}`,
      'Content-Type':
'application/json'
    },
    body: JSON.stringify({
student_id: student.id })
  })


revalidatePath('/dashboard/students')
  return { success: true, student }
}
```

---

10. ARQUITETURA REACT NATIVE — APP DO ALUNO

10.1 Estado Global da Sessão Ativa

```
// src/store/workout-session.store.ts
import { create } from 'zustand'
import type { WorkoutExercise,
```

```
WorkoutPlan } from
'@/lib/types/database.types'

interface SetLog {
  set_number: number
  load_used: string
  reps_done: string
  is_completed: boolean
}

interface SessionState {
  sessionId: string | null
  plan: WorkoutPlan | null
  currentExerciseIndex: number
  exerciseLogs: Record<string,
SetLog[]> // keyed by
workout_exercise_id
  startedAt: Date | null
  isActive: boolean

  // Actions
  startSession: (sessionId: string,
plan: WorkoutPlan) => void
  logSet: (workoutExerciseId: string,
setLog: SetLog) => void
  nextExercise: () => void
  prevExercise: () => void
  finishSession: () => void
  reset: () => void
```

```
}

export const useWorkoutSession =
create<SessionState>((set, get) => ({
  sessionId: null,
  plan: null,
  currentExerciseIndex: 0,
  exerciseLogs: {},
  startedAt: null,
  isActive: false,

  startSession: (sessionId, plan) =>
set({
    sessionId,
    plan,
    currentExerciseIndex: 0,
    exerciseLogs: {},
    startedAt: new Date(),
    isActive: true
  }),

  logSet: (workoutExerciseId, setLog)
=> set(state => ({
    exerciseLogs: {
      ...state.exerciseLogs,
      [workoutExerciseId]: [
        ...
(state.exerciseLogs[workoutExerciseId
] ?? []).filter(
```

```
          s => s.set_number !==
setLog.set_number
        ),
        setLog
      ]
    }
  })),

  nextExercise: () => set(state => ({
    currentExerciseIndex: Math.min(
      state.currentExerciseIndex + 1,

(state.plan?.workout_exercises?.lengt
h ?? 1) - 1
    )
  })),

  prevExercise: () => set(state => ({
    currentExerciseIndex:
Math.max(state.currentExerciseIndex -
1, 0)
  })),

  finishSession: () => set({
isActive: false }),
  reset: () => set({
    sessionId: null, plan: null,
currentExerciseIndex: 0,
    exerciseLogs: {}, startedAt:
```

```
null, isActive: false
  })
}))
```

---

11. COMPONENTES DE UI — DESIGN SYSTEM

11.1 Tokens de Design

```
// Paleta de cores (baseada no
MFitPersonal, adaptada)
export const colors = {
  // Primárias
  primary:       '#1B6EE5',   // Azul
principal
  primaryDark:   '#1456B5',   //
Hover / pressed
  primaryLight:  '#EBF2FF',   //
Background suave

  // Secundárias
  success:       '#22C55E',   //
Botão INICIAR (verde)
  warning:       '#F59E0B',   //
Alertas
  danger:        '#EF4444',   //
```

```
Erros / vencido
  info:          '#0EA5E9',   //
Informativo

  // Neutros
  dark:          '#1E293B',   //
Header / texto principal
  darkMid:       '#334155',   //
Texto secundário
  mid:           '#64748B',   //
Texto terciário
  light:         '#E2E8F0',   //
Bordas
  lighter:       '#F8FAFC',   //
Background cards
  white:         '#FFFFFF',

  // Superficie
  surface:       '#FFFFFF',
  background:    '#F1F5F9',
  headerBg:      '#1E2D40',   //
Header escuro do app
}

// Tipografia
export const typography = {
  fontFamily: {
    sans: 'Inter, system-ui, sans-
serif',
```

```
    mono: 'JetBrains Mono, monospace'
  },
  fontSize: {
    xs:   '12px',
    sm:   '14px',
    base: '16px',
    lg:   '18px',
    xl:   '20px',
    '2xl':'24px',
    '3xl':'30px',
  },
  fontWeight: {
    regular: '400',
    medium:  '500',
    semibold:'600',
    bold:    '700',
  }
}

// Espaçamento (múltiplos de 4px)
export const spacing = {
  1: '4px',   2: '8px',   3: '12px',
4: '16px',
  5: '20px',  6: '24px',  8: '32px',
10: '40px',
  12: '48px', 16: '64px'
}

// Border radius
```

```
export const radius = {
  sm: '6px', md: '10px', lg: '14px',
xl: '20px', full: '9999px'
}
```

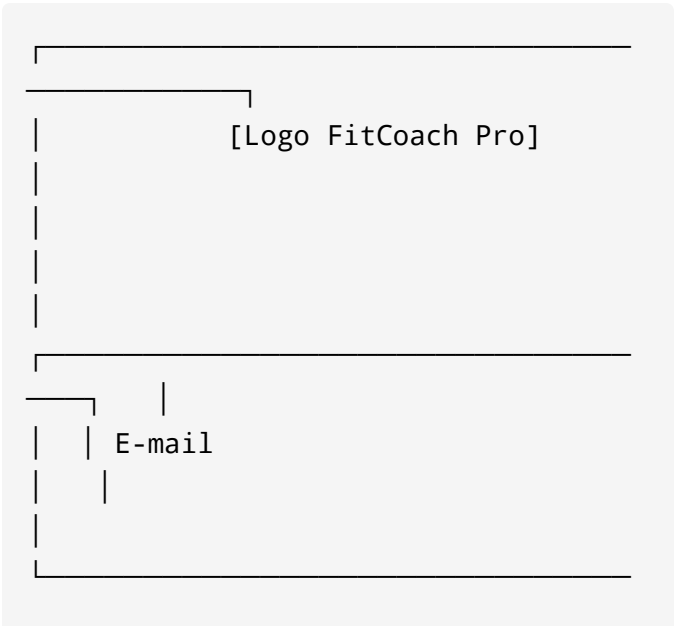
---

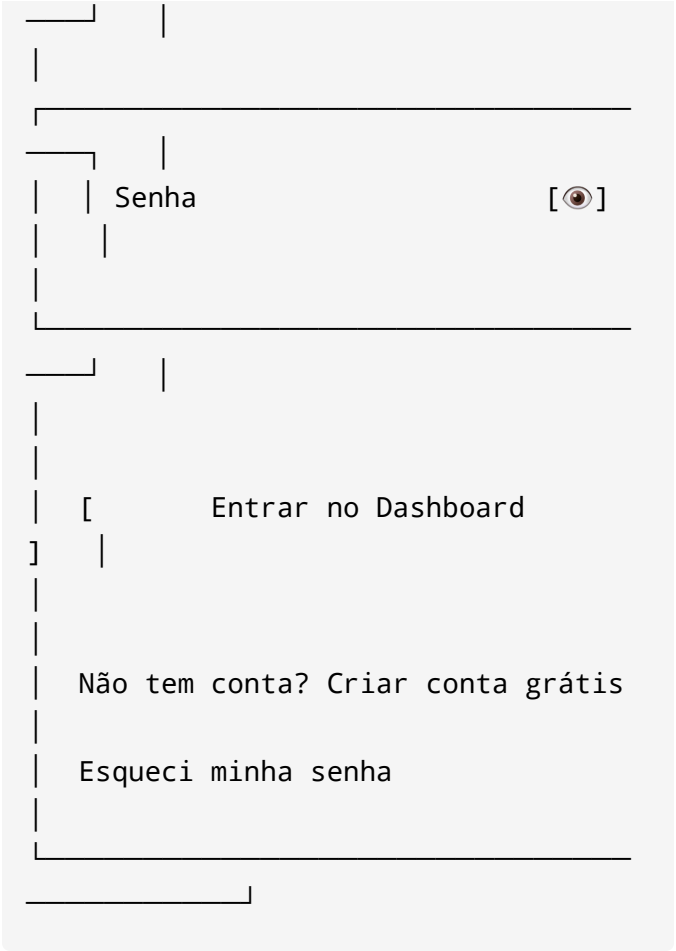
12. ESPECIFICAÇÃO DE TELAS — DASHBOARD (WEB)

TELA: Login do Personal

Rota: `/login`

Componentes: `LoginForm`, `Logo`





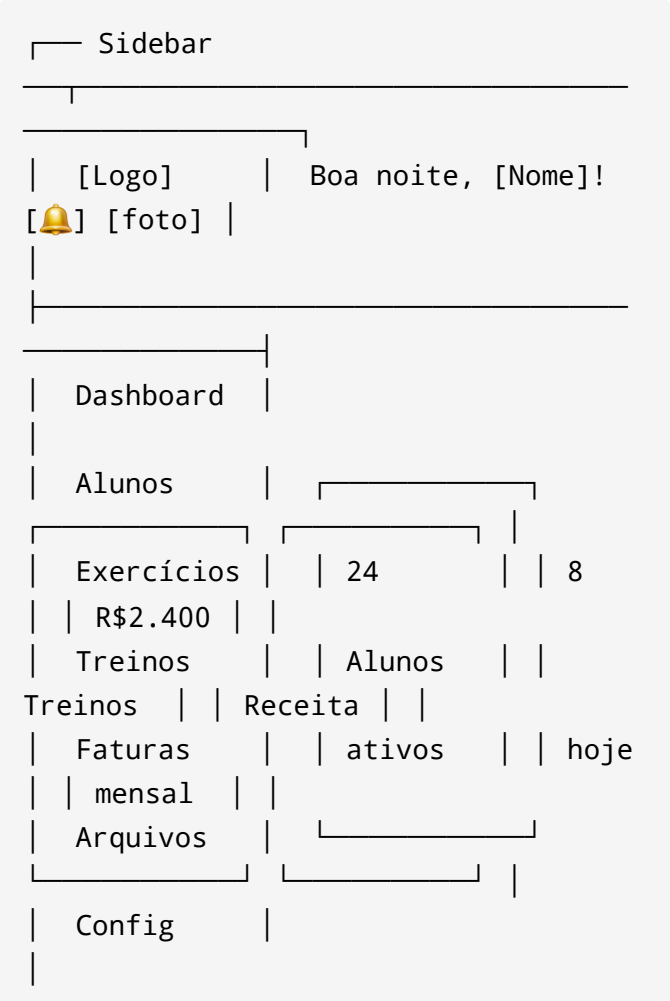
Validações:

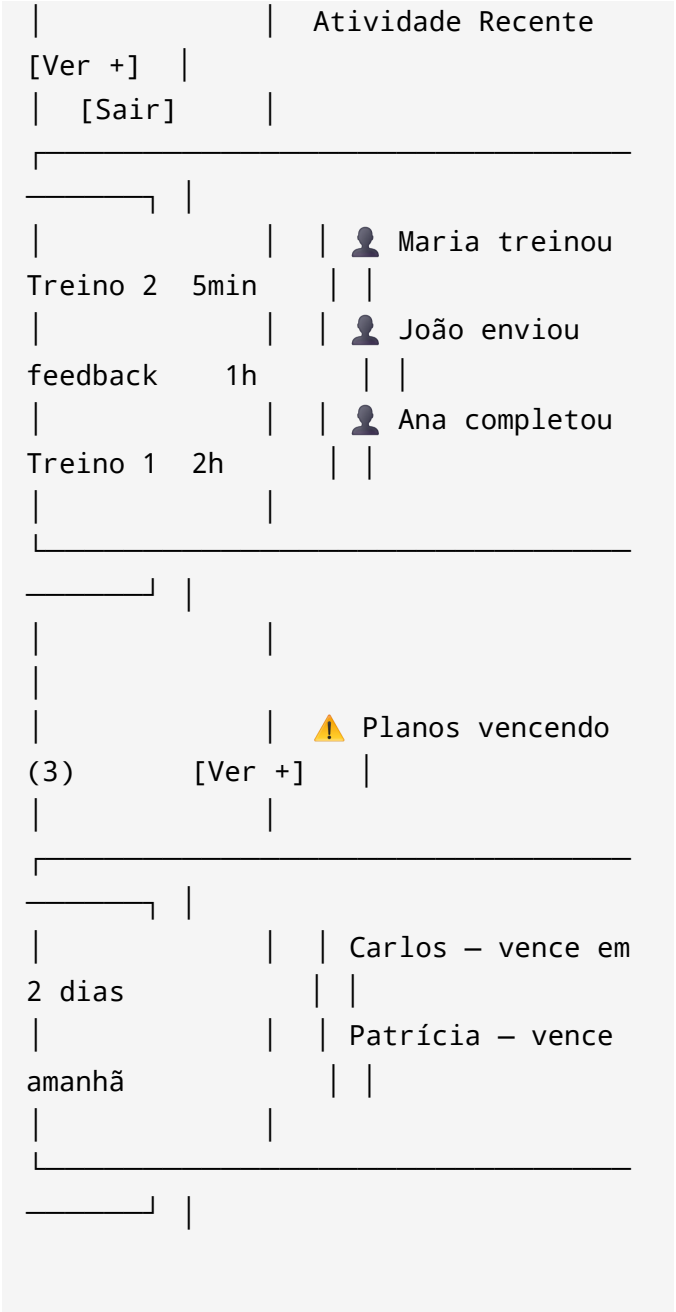
- E-mail: formato válido, obrigatório
- Senha: mínimo 8 caracteres
- Erro: "E-mail ou senha incorretos"
- Loading state no botão durante submit

TELA: Home do Dashboard

Rota: `/dashboard`

Data fetching: Server Component com `PersonalService.getOverview()`




Atividade Recente
[Ver +]
[Sair]
Maria treinou Treino 2 5min
João enviou feedback 1h
Ana completou Treino 1 2h
Planos vencendo (3)
[Ver +]
Carlos — vence em 2 dias
Patrícia — vence amanhã

```
└──────────────┴──────────────────────
──────────────────────────────────┘
```

TELA: Gestão de Alunos

Rota: `/dashboard/students`

```
Alunos
[+ Novo Aluno]
──────────────────────────────────────
──────────────────────────
[🔍 Buscar aluno...]    [Todos ▾]
[Ativos ▾]

┌──────────────────────────────────────
──────────────────────────┐
│ [foto] Maria Silva
│
│        Objetivo: Emagrecer
│
│        Último treino: hoje
│
│        Plano: Ativo até 30/06
│
│        [Editar] [Treinos]
[WhatsApp] [Inativar]    │
├──────────────────────────────────────
──────────────────────────┤
```

```
│ [foto] João Santos
│
│        Objetivo: Ganhar massa
│
│        Último treino: há 5 dias  ⚠️
│
│        Plano: Vencido            🔴
│
│        [Editar] [Treinos]
[WhatsApp] [Inativar]    │
└──────────────────────────────────────
──────────────────────────┘
```

TELA: Editor de Treino

Rota:

`/dashboard/students/[id]/workouts/[planId]`

```
← Voltar   Treino 1 — Maria Silva
[Publicar] [Salvar]

Nome do treino: [Treino 1
]
Descrição:      [Peito / Tríceps /
Ombros]
Status: ● Rascunho
```

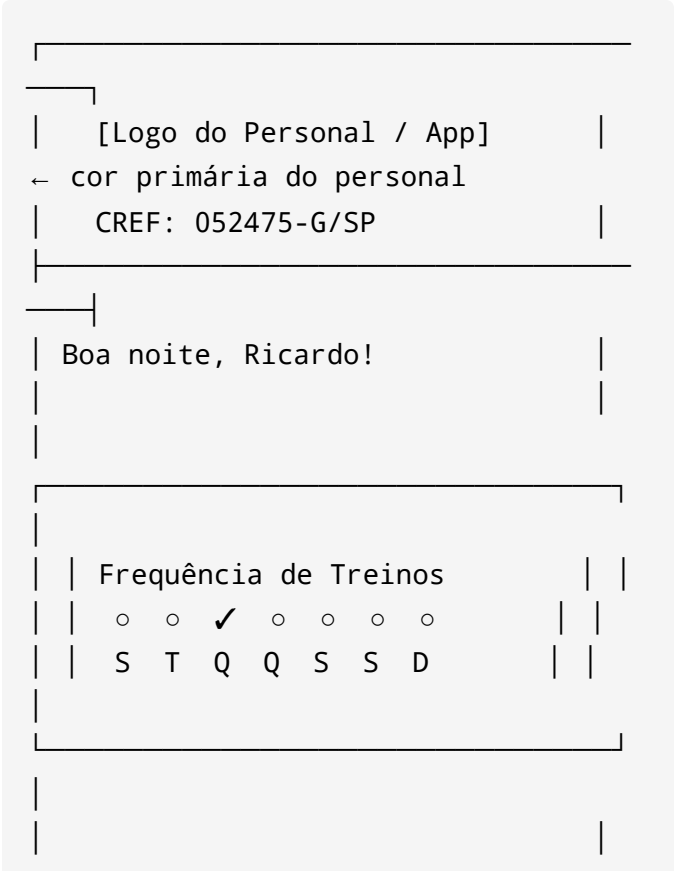
```
_____
_____
Exercícios
[+ Adicionar]
_____
_____
≡  1. Polichinelo
      Séries: [3x]  Reps: [30]
Carga: [0kg]  Int: [30s]
      Instruções: [
]
      Vídeo: [🎬 exercise-thumb.jpg]
[Trocar vídeo]
      [Remover]

≡  2. Supino Reto com Barra Reta
      Séries: [4x]  Reps: [10 a 12]
Carga: [10/10]  Int: [45s]
      Instruções: [Cuidado com a
carga! Priorize...  ]
      Vídeo: [🎬 supino-thumb.jpg]
[Trocar vídeo]
      [Remover]

[+ Adicionar exercício do banco]
```

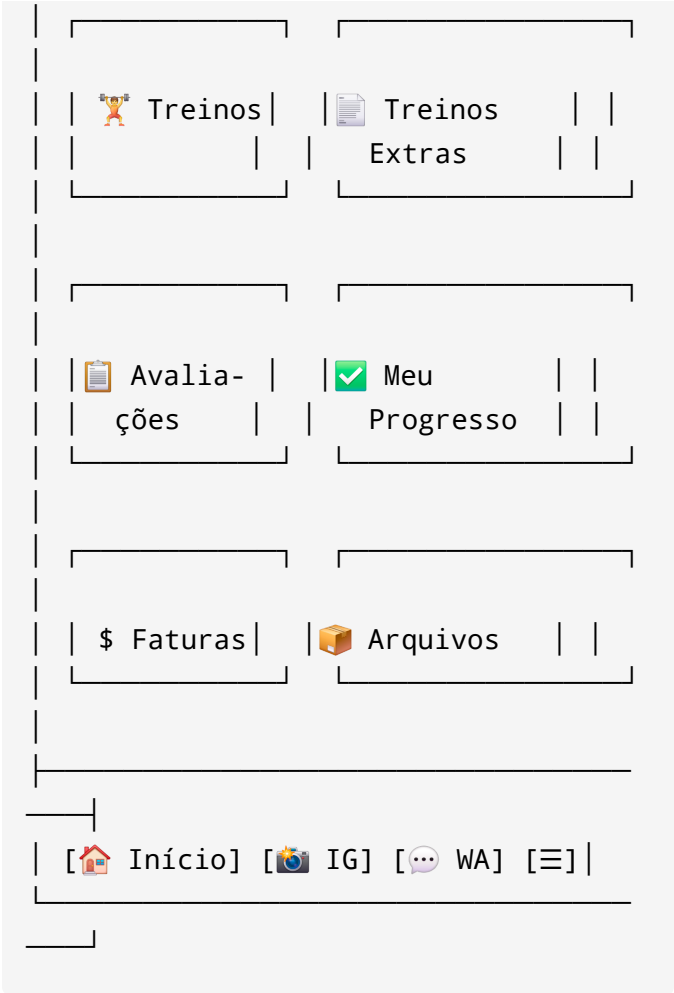
13. ESPECIFICAÇÃO DE TELAS — APP MOBILE (ALUNO)

TELA: Home do Aluno

Rota: `/` (index)

Componentes: `Header`, `WeeklyFrequency`, `QuickActions`, `BottomNav`





Lógica da Frequência Semanal:

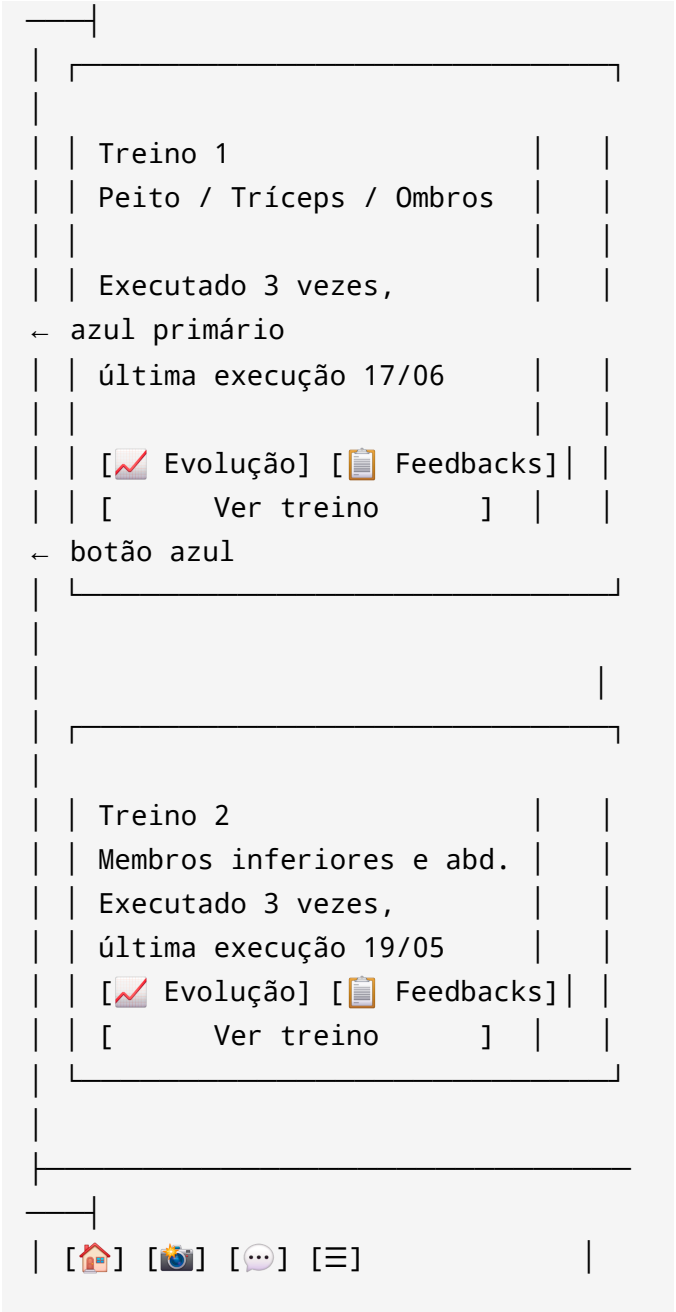
```
// Buscar sessões da semana atual
(domingo a sábado)
const startOfWeek =
getStartOfWeek(new Date())  //
```

```
domingo
const endOfWeek = getEndOfWeek(new
Date())      // sábado

const { data: sessions } = await
supabase
  .from('workout_sessions')
  .select('started_at')
  .eq('student_id', studentId)
  .eq('is_completed', true)
  .gte('started_at',
startOfWeek.toISOString())
  .lte('started_at',
endOfWeek.toISOString())

// Mapear dias com treino (0=Dom ...
6=Sab)
const trainedDays = sessions.map(s =>
new Date(s.started_at).getDay())
```

TELA: Lista de Treinos

Rota: `/workouts`

```
┌──────────────────────────────────────
────┐
│       [Logo]                       │
├──────────────────────────────────────
```

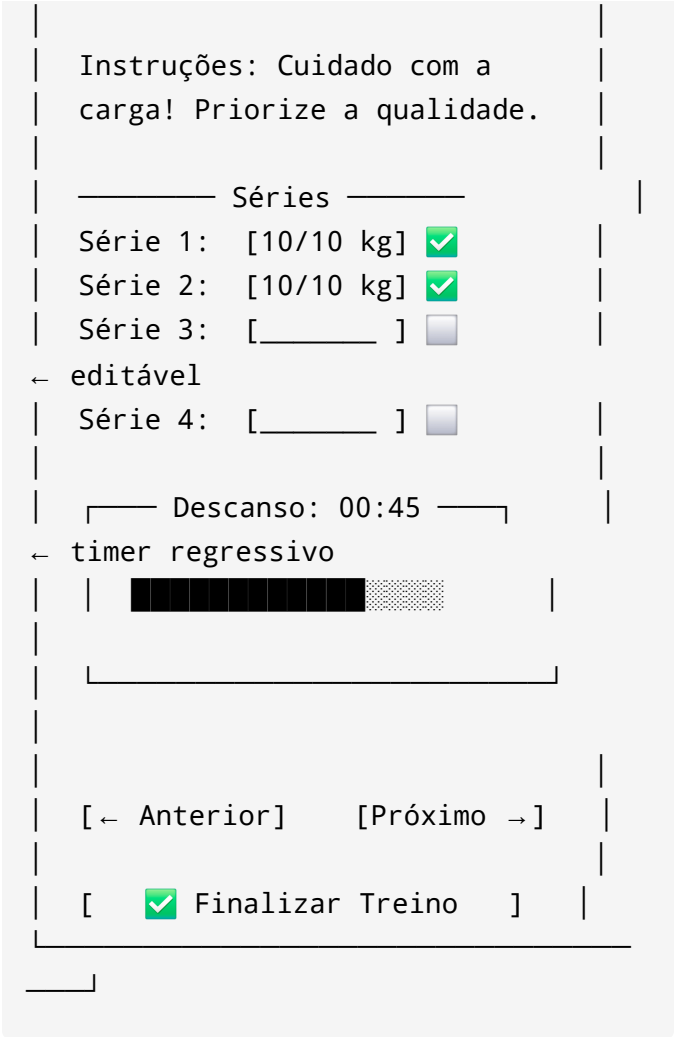

Treino 1
Peito / Tríceps / Ombros
Executado 3 vezes,
azul primário
última execução 17/06
[📈 Evolução]
[📋 Feedbacks]
[Ver treino]
botão azul
Treino 2
Membros inferiores e abd.
Executado 3 vezes,
última execução 19/05
[📈 Evolução]
[📋 Feedbacks]
[Ver treino]
[🏠] [📸] [💬] [☰]

```
└──────────────────────────────────────────
────┘
```

TELA: Treino em Execução (Modo Ativo)

Rota: `/workouts/[id]/active`
State: Zustand `useWorkoutSession`

```
┌──────────────────────────────────────────
────┐
│  ← Sair    Treino 1    2/5 📊   │
│  ████████░░░░░░░░  40%
│  ← progress bar
├──────────────────────────────────────────
────┤
│                                  │
│  Supino Reto com Barra Reta      │
← nome do exercício
│  Peito                           │
← grupo muscular
│                                  │
│  ┌──────────────────────────────┐
│
│  │      [▶ Vídeo]          │   │
← player inline
│  └──────────────────────────────┘
│
```



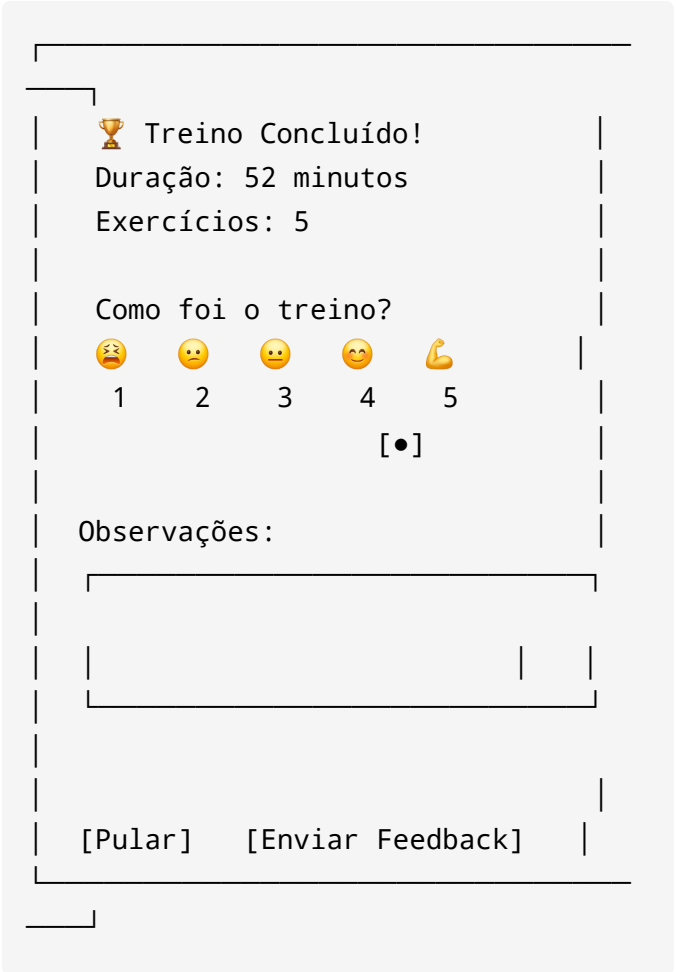
Comportamento do Timer:

- Ao marcar uma série como concluída: timer inicia automaticamente
- Timer toca som/vibra ao chegar em 0

- Usuário pode pausar / pular descanso

TELA: Feedback Pós-Treino

Modal após finalizar



14. FLUXOS DE AUTENTICAÇÃO

Fluxo 1: Cadastro do Personal

```
1. Personal acessa /register
2. Preenche: nome, e-mail, senha,
CREF
3. supabase.auth.signUp({ email,
password, data: { role: 'personal',
name } })
4. Trigger `on_personal_signup` →
cria registro em `personals`
5. E-mail de confirmação enviado
6. Personal confirma e-mail
7. Redirect para
/dashboard/onboarding (configurar
perfil)
```

Fluxo 2: Convite e Login do Aluno

```
1. Personal cria aluno no dashboard
2. Sistema chama Edge Function
`invite-student`
3. Edge Function cria auth.user com
senha temporária
4. Trigger `on_student_signup` →
```

```
vincula student ao user_id
5. E-mail enviado com credenciais
temporárias
6. Aluno abre app → login com e-mail
+ senha temporária
7. App detecta `must_change_password
= true` no user metadata
8. Redirect para /change-password
9. Aluno define nova senha
10. Atualiza metadata:
`must_change_password = false`
11. Redirect para Home
```

Fluxo 3: Refresh Token

```
// No app mobile — interceptar erro
401 e renovar token
supabase.auth.onAuthStateChange((even
t, session) => {
  if (event === 'TOKEN_REFRESHED') {
    // Atualizar store local com novo
token
  }
  if (event === 'SIGNED_OUT') {
    // Limpar store e redirecionar
para login
    router.replace('/login')
```

```
  }
})
```

---

15. CRON JOBS E AUTOMAÇÕES

```
# supabase/config.toml — Cron Jobs (pg_cron)

[cron]
# Atualizar faturas vencidas — todo dia às 01:00
"0 1 * * *" = "SELECT net.http_post(url:='https://seu-projeto.supabase.co/functions/v1/overdue-invoices', headers:='{\"Authorization\": \"Bearer SERVICE_KEY\"}'::jsonb)"

# Lembrete de treino — todo dia às 08:00
"0 8 * * *" = "SELECT net.http_post(url:='https://seu-projeto.supabase.co/functions/v1/workout-reminder',
```

```
headers:='{\"Authorization\":
\"Bearer SERVICE_KEY\"}'::jsonb)"
```

Automação: Aluno sem treinar há 5 dias

```
-- View usada pelo cron de alerta
CREATE OR REPLACE VIEW
public.inactive_students_alert AS
SELECT
  s.id AS student_id,
  s.name,
  s.personal_id,
  MAX(ws.started_at) AS
last_workout_at,
  EXTRACT(DAY FROM NOW() -
MAX(ws.started_at)) AS days_inactive
FROM public.students s
LEFT JOIN public.workout_sessions ws
  ON ws.student_id = s.id AND
ws.is_completed = true
WHERE s.is_active = true
GROUP BY s.id, s.name, s.personal_id
HAVING
  MAX(ws.started_at) < NOW() -
INTERVAL '5 days'
  OR MAX(ws.started_at) IS NULL;
```

---

16. VARIÁVEIS DE AMBIENTE

fitcoach-pro-web (.env.local)

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxxx
x.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...  #
Nunca expor no client

# App
NEXT_PUBLIC_APP_URL=https://fitcoachp
ro.vercel.app

# E-mail (Resend)
RESEND_API_KEY=re_...

# Pagamentos (futuro)
STRIPE_SECRET_KEY=sk_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

fitcoach-pro-mobile (.env)

```
EXPO_PUBLIC_SUPABASE_URL=https://xxxx
x.supabase.co
```

```
EXPO_PUBLIC_SUPABASE_ANON_KEY=eyJ...
EXPO_PUBLIC_APP_SCHEME=fitcoachpro
```

Supabase Edge Functions (Secrets)

```
supabase secrets set
RESEND_API_KEY=re_...
supabase secrets set
WHATSAPP_TOKEN=...
```

---

17. PADRÕES DE CÓDIGO E CONVENÇÕES

Naming

```
Arquivos:        kebab-case
→ student-card.tsx
Componentes:     PascalCase
→ StudentCard
Hooks:           camelCase + use
→ useStudents
Services:        PascalCase +
Service→ StudentService
Tabelas DB:      snake_case plural
→ workout_plans
```

```
Colunas DB:      snake_case
→ personal_id
Types:           PascalCase
→ WorkoutPlan
Enums DB:        snake_case
→ muscle_group
```

Estrutura de Componente

```
// Sempre na ordem:
// 1. Imports externos
// 2. Imports internos
// 3. Types / interfaces locais
// 4. Componente
// 5. Export default

import { useState } from 'react'
import { Button } from
'@/components/ui/button'
import { StudentService } from
'@/lib/services/student.service'
import type { Student } from
'@/lib/types/database.types'

interface StudentCardProps {
  student: Student
  onEdit: (id: string) => void
}
```

```
export function StudentCard({
student, onEdit }: StudentCardProps)
{
  // ...
}
```

Tratamento de Erros

```
// SEMPRE desestruturar { data, error
} do Supabase
const { data: students, error } =
await StudentService.list(personalId)

if (error) {

console.error('[StudentService.list]'
, error)
  throw new Error('Erro ao carregar
alunos')
}

// Nunca fazer:
const students = await
supabase.from('students').select('*')
// sem verificar error
```

---

18. TESTES

Estratégia de Testes

Tipo	Framework	Cobertura
Unit (Services)	Vitest	Service Layer completo
Integration (API)	Vitest + Supabase local	Fluxos críticos
E2E (Web)	Playwright	Login, criar aluno, criar treino
E2E (Mobile)	Detox	Login aluno, executar treino

Exemplo — Teste de Service

```
//
src/lib/services/__tests__/student.se
rvice.test.ts
import { describe, it, expect,
```

```
beforeEach } from 'vitest'
import { StudentService } from
'../student.service'

describe('StudentService', () => {
  it('should create a student with
required fields', async () => {
    const { data, error } = await
StudentService.create({
      personal_id: TEST_PERSONAL_ID,
      name: 'Maria Silva',
      email: 'maria@test.com',
    })

    expect(error).toBeNull()
    expect(data).toMatchObject({
      name: 'Maria Silva',
      is_active: true,
      training_days: []
    })
  })

  it('should not allow duplicate
email for same personal', async () =>
{
    const { error } = await
StudentService.create({
      personal_id: TEST_PERSONAL_ID,
      name: 'Duplicado',
```

```
      email: 'maria@test.com', // já
existe
    })

    expect(error).not.toBeNull()
  })
})
```

---

19. CI/CD — GITHUB + VERCEL

GitHub Actions

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Type check
        run: npx tsc --noEmit

      - name: Lint
        run: npm run lint

      - name: Unit tests
        run: npm run test

      - name: Generate Supabase types
        run: npx supabase gen types
typescript --project-id ${{
secrets.SUPABASE_PROJECT_ID }} >
src/lib/types/database.types.ts

  deploy:
    needs: test
    if: github.ref ==
'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
```

```
      - name: Deploy to Vercel
        run: npx vercel --prod --
token ${{ secrets.VERCEL_TOKEN }}
```

Branches

```
main          → Produção (Vercel
deploy automático)
develop       → Staging (Vercel
preview)
feature/xyz   → Features (PR →
develop)
fix/xyz       → Correções (PR →
develop)
```

Supabase Migrations em CI

```
- name: Run Supabase migrations
  run: |
    npx supabase db push --project-
ref ${{ secrets.SUPABASE_PROJECT_ID
}}
  env:
    SUPABASE_ACCESS_TOKEN: ${{
secrets.SUPABASE_ACCESS_TOKEN }}
```

---

20. CHECKLIST SDD POR SPRINT

✅ Sprint 1 — Fundação

Schema:

- [ ] Migration 001: personals
- [ ] Migration 002: students
- [ ] Migration 010: RLS base (personals + students)
- [ ] Migration 011: Triggers (handle_new_personal, link_student_to_user)
- [ ] `supabase gen types typescript` → database.types.ts commitado

Backend:

- [ ] Edge Function: invite-student
- [ ] Storage Bucket: avatars

Web:

- [ ] Setup Next.js 14 + TypeScript + ESLint
- [ ] Integração Supabase (client + server + middleware)

- [ ] Tela: Login do Personal
- [ ] Tela: Registro do Personal
- [ ] Tela: Dashboard Home (esqueleto)
- [ ] Layout com Sidebar

Mobile:

- [ ] Setup Expo + Expo Router
- [ ] Integração Supabase
- [ ] Tela: Login do Aluno
- [ ] Tela: Trocar Senha (primeiro acesso)
- [ ] Tela: Home (esqueleto com WeeklyFrequency)

✅ Sprint 2 — Alunos e Treinos (Preview)

Schema:

- [ ] Migration 003: exercises
- [ ] Migration 004: workout_plans + workout_exercises
- [ ] Migration 010: RLS exercises + workout_plans
- [ ] View: workout_plan_stats

- [ ] Regenerar types

Web:

- [ ] Tela: Lista de Alunos
- [ ] Tela: Criar/Editar Aluno
- [ ] Tela: Banco de Exercícios (CRUD)
- [ ] Tela: Criar Treino + Editor de Exercícios
- [ ] Funcionalidade: Publicar Treino

Mobile:

- [ ] Tela: Lista de Treinos
- [ ] Tela: Ver Treino (Preview Mode)
- [ ] Player de vídeo (YouTube + upload)
- [ ] BottomNav completo

✅ Sprint 3 — Execução do Treino

Schema:

- [ ] Migration 005: workout_sessions + session_exercise_logs
- [ ] Migration 010: RLS sessions + logs
- [ ] View: student_weekly_frequency
- [ ] Regenerar types

Mobile:

- [ ] Store Zustand: workout-session
- [ ] Tela: Treino em Execução (Modo Ativo)
- [ ] Componente: SeriesTracker (marcar séries + editar carga)
- [ ] Componente: RestTimer (countdown + som)
- [ ] Tela: Modal Feedback Pós-Treino
- [ ] Frequência semanal na Home (dados reais)
- [ ] Tela: Evolução do Treino (gráfico de cargas)

✅ Sprint 4 — Avaliações e Progresso

Schema:

- [ ] Migration 006: physical_assessments + student_progress
- [ ] RLS assessments + progress
- [ ] Storage Bucket: assessment-photos
- [ ] Regenerar types

Web:

- [ ] Tela: Criar Avaliação Física
- [ ] Upload de fotos de avaliação (3 ângulos)
- [ ] Tela: Histórico de Avaliações do Aluno

Mobile:

- [ ] Tela: Avaliações (histórico + gráficos)
- [ ] Tela: Meu Progresso (timeline de fotos + gráfico de peso)
- [ ] Upload de foto de progresso pelo aluno

✅ Sprint 5 — Financeiro, Arquivos e Notificações

Schema:

- [ ] Migration 007: invoices
- [ ] Migration 008: files
- [ ] Migration 009: notifications + push_tokens
- [ ] Cron Job: overdue-invoices
- [ ] Edge Function: workout-reminder
- [ ] Storage Bucket: student-files
- [ ] Regenerar types

Web:

- [ ] Tela: Gestão de Faturas (lista + criar + marcar pago)
- [ ] Tela: Arquivos (upload + associar aluno)
- [ ] Relatório financeiro mensal

Mobile:

- [ ] Tela: Faturas do Aluno
- [ ] Tela: Arquivos do Aluno (visualizar PDF)
- [ ] Push Notifications (Expo)
- [ ] Centro de Notificações in-app

✅ Sprint 6 — White-label e Deploy

- [ ] Configurações White-label (cor, logo, nome do app)
- [ ] Aplicar cor primária dinamicamente no app mobile
- [ ] Tela de Configurações do Personal (web)
- [ ] Testes E2E Playwright (web)
- [ ] Testes Detox (mobile)
- [ ] GitHub Actions CI/CD configurado

- [ ] Variáveis de ambiente em produção (Vercel + Supabase)
- [ ] Deploy de produção

SPEC gerada em 17/06/2026 | Metodologia: Schema-Driven Development | Referência: PRD v1.0